Python - Fonctions de hachage

November 8, 2026

Ce TP a pour objectif d'identifier des collisions dans des fonctions de hachage cryptographiques classiques, en étudiant leurs comportements lorsque leur sortie est tronquée (limitée à un nombre réduit de bits). Nous explorerons différentes approches méthodologiques en faisant varier le nombre de bits de sortie, puis nous analyserons et comparerons les résultats obtenus.

Rappel : les fonctions de hachage cryptographiques

Définition. Une fonction de hachage H associe à un message m de taille *quelconque* une *empreinte* (ou *haché*, en anglais *digest*) $H(m)$ de taille *fixe*, égale à n bits :

$$H : \{0,1\}^* \longrightarrow \{0,1\}^n$$

Exemples : MD5 ($n = 128$), SHA-1 ($n = 160$), SHA-256 ($n = 256$, famille SHA-2), SHA-3.

À quoi servent-elles ?

- **Intégrité** : on vérifie qu'un fichier n'a pas été modifié en comparant son empreinte à celle publiée par son auteur.
- **Signature numérique** : on signe l'empreinte $H(m)$, courte, plutôt que le message m tout entier.
- **Mots de passe** : un serveur stocke l'empreinte (salée) d'un mot de passe, jamais le mot de passe lui-même.
- Mais aussi : dérivation de clés, authentification de messages (HMAC), signatures post-quantiques (SLH-DSA)...

Propriétés attendues. H doit être déterministe et rapide à calculer. Une bonne fonction de hachage se comporte comme une fonction aléatoire : modifier un seul bit de m change en moyenne la moitié des bits de $H(m)$ (*effet avalanche*, analogue à la diffusion de Shannon vue en cours). Surtout, elle doit vérifier :

1. **Résistance à la préimage** (fonction *à sens unique*) : étant donné une empreinte h, il est difficile de trouver un message m tel que $H(m) = h$.
2. **Résistance à la seconde préimage** : étant donné un message m, il est difficile de trouver un autre message $m' \neq m$ tel que $H(m') = H(m)$.
3. **Résistance aux collisions** : il est difficile de trouver deux messages quelconques $m \neq m'$ tels que $H(m) = H(m')$.

Il y a une infinité de messages, mais seulement 2^n empreintes possibles : des collisions **existent** donc toujours. On demande seulement qu'elles soient impossibles à **trouver** en pratique. Une fonction de hachage est dite *cassée* dès que l'on sait en trouver nettement plus vite que par recherche exhaustive : c'est le cas de MD5 (2004) et de SHA-1 (2017).

Partie 1 - Calcul d'un hash La bibliothèque Python `hashlib` fournit une implémentation standard des fonctions de hachage cryptographiques (https://docs.python.org/3/library/hashlib.html).

1. Parcourez la documentation de la bibliothèque hashlib afin de déterminer une méthode pour calculer le digest SHA2 d'un message et implémentez là.

2. Adaptez ce code afin qu'il tronque la sortie aux 5 premiers bits du digest et affichez-le.

3. Récupérez et stockez la liste de mots contenus dans le fichier https://raw.githubusercontent.com/chrplr/openlexicon/master/datasets-info/Liste-de-mots-francais-Gutenberg/liste.de.mots.francais.frgut.txt et calculez le hash du premier de ces mots.

Partie 2 - Effet avalanche

1. Implémentez une fonction `inverse_bit(message, i)` qui inverse le i-ème bit d'un message de type `bytes`, et une fonction `distance_hamming(x, y)` qui compte le nombre de bits qui diffèrent entre deux empreintes. On pourra convertir les empreintes en entiers avec `int.from_bytes` et utiliser l'opérateur `^` (ou exclusif).

2. Comparez l'empreinte SHA-256 de `b"INSA"` à celle du même message dont on a inversé un seul bit. Combien de bits ont changé ?

3. Répétez l'expérience sur 1000 mots de la liste, en inversant à chaque fois un bit choisi au hasard, et tracez l'histogramme des distances obtenues. Quelle valeur moyenne observez-vous ? Interprétez ce résultat à l'aide de l'encadré.

Partie 3 - Sécurité d'un hash

1. Calculez le hash MD5 des fichiers `order.ps` et `letter.ps` à l'aide de Python et affichez les résultats.

2. Calculez ensuite le hash SHA2 de ces deux fichiers.

3. Comparez les valeurs obtenues avec MD5 et SHA2. Que constatez-vous ? Quelle conclusion pouvez-vous en tirer concernant la sécurité de MD5 par rapport à SHA2 ?

Partie 4 - Détermination d'une collision

1. Première méthode

   ◇ Calculez le haché MD5 tronqué à 5 bits du mot "INSA".

   ◇ Itérez sur la liste de mots de la partie 1 jusqu'à ce qu'un hash tronqué identique soit trouvé et indiquez qu'une collision a été trouvée.

   ◇ Affichez ces mots et le nombre d'itérations avant de les avoir trouvés.

   ◇ Même chose avec le mot "intergouvernementalisation"

2. Seconde méthode

   ◇ Ecrivez un programme qui:

   ◇ Choisit deux mots distincts de la liste de mots

   ◇ Calcule leur hash tronqué

   ◇ Retourne les mots choisis si une collision se réalise, ainsi que le nombre d'itérations.

3. Troisième méthode : l'attaque des anniversaires

   ◇ Parcourez la liste de mots en mémorisant chaque haché déjà rencontré dans un dictionnaire Python (clé : le haché tronqué, valeur : le mot).

   ◇ Arrêtez-vous dès que le haché du mot courant figure déjà dans le dictionnaire : vous tenez une collision.

   ◇ Affichez les deux mots et le nombre d'itérations.

4. Comparez ces trois méthodes. Laquelle cherche une seconde préimage, lesquelles cherchent une collision (voir l'encadré) ?

Partie 5 - Impact de la taille du digest

1. Modifiez la fonction de calcul de digest de sorte qu'elle prenne la taille désirée du digest en tant que paramètre de la fonction.

2. Exécutez chacune des méthodes pour des tailles de digest allant de 6 à 15 et relevez le nombre d'itérations utilisées. Pour obtenir des résultats fiables, répétez chaque mesure une vingtaine de fois en mélangeant la liste (`random.shuffle`) et faites la moyenne. Poursuivez jusqu'à 30 bits pour la troisième méthode.

3. Tracez sur un même graphique (`matplotlib`) le logarithme en base 2 du nombre moyen d'itérations en fonction de la taille du digest, pour les trois méthodes. Quelle pente observez-vous pour chacune ? Qu'en déduisez-vous sur le coût de chaque attaque ?

4. Quelle est, selon vous, la probabilité théorique de trouver de telles collisions si le digest généré par la fonction de hachage garde ses 128 bits ?

5. À raison d'un milliard de hachés par seconde, combien de temps faudrait-il en moyenne pour trouver une collision sur MD5 complet ? Une seconde préimage ? Que signifie alors "MD5 est cassée", au vu de la partie 3 ?

Partie 6 - Paradoxe des anniversaires

1. Parcourez https://fr.wikipedia.org/wiki/Paradoxe_des_anniversaires et implémentez une fonction `anniversaire(n)` qui calcule la probabilité que deux personnes dans un groupe de n personnes partagent le même anniversaire.

2. Faites varier la taille du groupe et représentez les résultats dans un graphique.

3. Comparez avec les résultats obtenus à la partie précédente, en remplaçant les 365 jours de l'année par les 2^n valeurs possibles d'un haché tronqué à n bits.

4. Calculez `anniversaire(153)`. Si votre fonction retourne 1, réécrivez `anniversaire` de sorte à toujours pouvoir obtenir la probabilité exacte

5. Etendez la fonction `anniversaire` pour calculer la probabilité que m personnes parmi n partagent le même anniversaire.

Partie 7 - Stockage des mots de passe Un site web a subi une fuite de sa base de données. Le fichier `fuite.txt` contient, pour chaque utilisateur, une ligne `identifiant:empreinte`, où l'empreinte est le SHA-256 (en hexadécimal) de son mot de passe encodé en UTF-8.

1. Avant tout calcul, observez les empreintes de `bob` et de `hugo`. Que pouvez-vous en déduire ?

2. **Attaque par dictionnaire.** Calculez l'empreinte de chaque mot de la liste de la partie 1 et comparez-la à celles de la fuite. Combien de mots de passe retrouvez-vous ? Combien d'empreintes avez-vous dû calculer ? Ce calcul serait-il à refaire pour attaquer la fuite d'un autre site ?

3. Deux empreintes résistent à votre attaque. Que pouvez-vous en déduire sur la façon dont ces mots de passe ont été choisis ? Qu'est-ce qu'un bon mot de passe ?

4. Le site a corrigé sa base. Chaque ligne du fichier `fuite_salee.txt` est désormais de la forme

   `identifiant:sel:empreinte`

   Le *sel* est une valeur aléatoire de 16 octets propre à chaque utilisateur (en hexadécimal), et l'empreinte vaut SHA-256(sel ‖ mot de passe), c'est-à-dire le SHA-256 des octets du sel suivis de ceux du mot de passe.

   ◇ Les empreintes de `bob` et de `hugo` sont-elles encore identiques ?

   ◇ Reprenez l'attaque par dictionnaire. Combien d'empreintes devez-vous maintenant calculer ?

   ◇ Pourquoi un attaquant ne peut-il plus précalculer, une fois pour toutes, une table {`empreinte: mot`} ?

5. **Hachage lent.** Mesurez (module `time`) le temps moyen d'un calcul de `hashlib.sha256`, de `hashlib.pbkdf2_hmac("sha256", mdp, sel, 600000)` et, s'il est disponible dans votre version de Python, de `hashlib.scrypt(mdp, salt=sel, n=2**14, r=8, p=1)`. Estimez dans chaque cas le temps nécessaire pour tester toute la liste de mots contre un seul utilisateur. Pourquoi ce ralentissement ne gêne-t-il pas l'utilisateur légitime ?